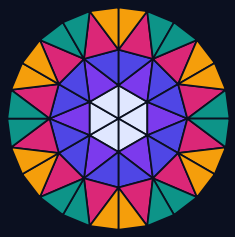
FLUTTER PACKAGE · iOS AND ANDROID



kaleidoscope

The complete integration guide

See every AI call your app makes: tokens, cost, speed and errors, by module, page and section. From first install to production, step by step.

Any AI model | Encrypted on device | Dashboard and forecasts | Beginner friendly

DEVELOPED BY

Sanket Santosh Vidya Manohar

Version 1.0.0
November 2026

BEFORE YOU BEGIN

About this guide

This guide takes you from an empty project to a fully tracked AI app. Part I sets everything up in about five minutes. Part II connects your AI calls, whichever provider or SDK you use. Parts III to V explain the dashboard, privacy, costs and advanced features, and Part VI is a reference to come back to.

Code you can copy is shown in dark boxes. Coloured boxes mark tips, notes, warnings and what you should see after each step.

GETTING STARTED · PART I

1 Welcome to kaleidoscope 2
2 Words you will see 4
3 Before you start 5
4 Install in five minutes 6

CONNECTING YOUR AI · PART II

5 Choose how to connect 8
6 Recipe: OpenAI with package:http 9
7 Recipe: Claude with Dio 11
8 Recipe: any SDK with track() 12
9 Recipe: streaming answers 13
10 Recipe: local and on-device models 14
11 Recipe: your backend and your own gateway 15

UNDERSTANDING USAGE · PART III

12 Tag calls by module, page and section 16
13 A tour of the dashboard 18
14 Using the dashboard in release builds 22
15 Debug console and floating overlay 23

PRIVACY, COST AND CONTROL · PART IV

16 Privacy and security 25
17 Costs, prices and budgets 27

GOING FURTHER · PART V

18 The analytics API 29
19 Build your own screens with chart widgets 31
20 Export and integrations 32
21 Provider fallback 34
22 Testing your app 35

REFERENCE · PART VI

23 Troubleshooting 36
24 Frequently asked questions 37
25 Reference 38
26 Checklist before you ship 40
27 Legal notices and disclaimer 41

kaleidoscope DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR

PART I · GETTING STARTED · CHAPTER 01

Welcome to kaleidoscope

IN THIS CHAPTER

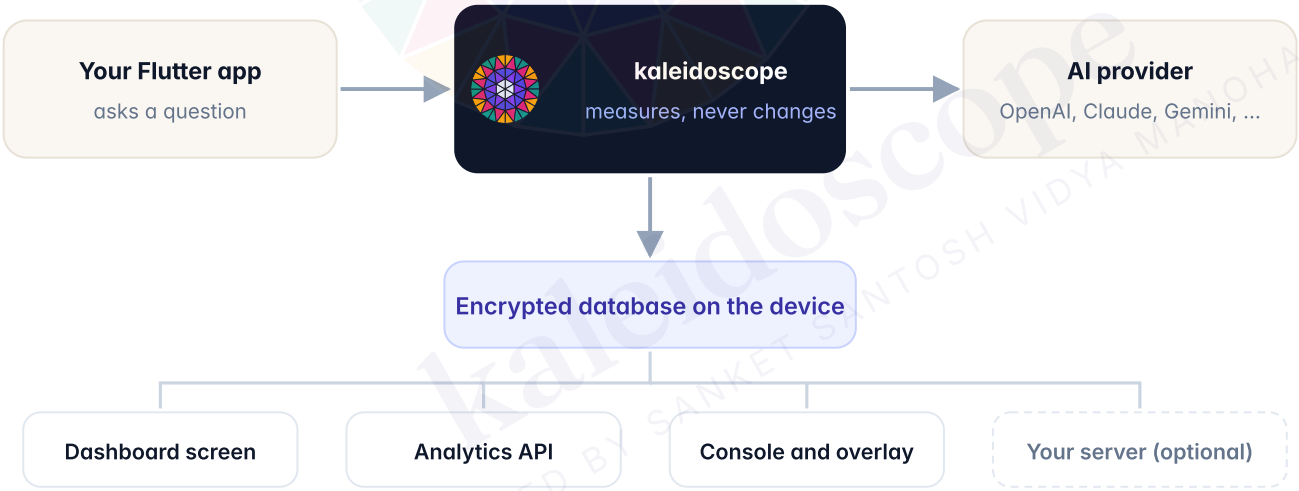
- What the package does, in plain words
- How it fits between your app and the AI
- What you get out of the box
- Who this guide is for

AI features cost money every time they run, and they can be slow or fail in ways that are hard to see. **kaleidoscope** shows you exactly what happens on every AI call your Flutter app makes: which model answered, how many tokens it used, what it cost, how long it took, and **which part of your app** asked for it.

It works with any AI model: OpenAI, Claude, Gemini, Mistral, Groq, DeepSeek, OpenRouter, a local Ollama model, a model running on the phone itself, or your own company gateway. All data stays on the device, in an encrypted database, unless you choose to send it somewhere.

How it works

kaleidoscope sits quietly between your app and the AI provider. It reads the request and the response as they pass, but it never changes them. If anything inside kaleidoscope fails, your AI call still completes normally.



What you get

Every call recorded
Model, provider, parameters, input, output, cached and reasoning tokens, cost, speed and errors.

Where it happened
Tag calls by module, page and section of your app, plus user, feature or anything custom.

Ready-made dashboard
One line opens a full analytics screen with charts, forecasts and a searchable call log.

Real costs

Built-in prices for 30+ models, including cache discounts. Update them any time, even remotely.

Private by default

Prompts are not stored unless you allow it. Encrypted storage, PII scrubbing and retention limits.

Easy debugging

Colour-coded console logs and a floating overlay while you develop. Hidden in release builds.

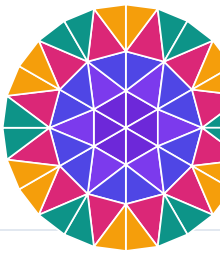
Who this guide is for

You only need to know basic Flutter: how to add a package, edit `main.dart` and run your app. Every step shows the exact code to paste and what you should see afterwards. Advanced chapters are marked, so you can skip them on your first read.

kaleidoscope
DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR

PART I · GETTING STARTED · CHAPTER 02

Words you will see



IN THIS CHAPTER

- The key terms used in this guide and in the dashboard
- How tokens relate to cost

You do not need to memorise these. Come back to this page whenever a word is unclear.

Term	Meaning
AI provider	The company or server that runs the model, such as OpenAI, Anthropic (Claude), Google (Gemini) or your own Ollama.
Model	The specific AI you call, for example `gpt-4o-mini`, `claude-sonnet-5-5` or `gemini-2.5-flash`. Each has its own price and speed.
Prompt	The text you send to the AI: the question, plus any instructions.
Response	What the AI sends back: the answer.
Token	The unit AI providers count and charge by. One token is roughly 4 characters of English text, so 1,000 tokens is about 750 words.
Input tokens	Tokens in your prompt. Usually cheaper.
Output tokens	Tokens in the answer. Usually 3 to 5 times more expensive than input tokens.
Cached tokens	Parts of the prompt the provider has seen recently and charges much less for. kaleidoscope shows how much caching saved you.
Latency	How long the whole call took, from sending to the last word of the answer.
Time to first token (TTFT)	For streamed answers: how long until the first word appeared. This is what the user feels as "speed".
Streaming	Receiving the answer word by word as it is generated, instead of all at once.
Module, page, section	Your own labels for where a call came from: module is a big area of your app (shop), page is a screen (cart), section is a part of the screen (suggestions).
Record	One saved entry describing one AI call.
Retention	How long records are kept before they are deleted automatically.
Redaction	Replacing private details such as emails or card numbers with placeholders before text is saved.
Sink	An optional destination that receives copies of your records, such as your server or an analytics service.

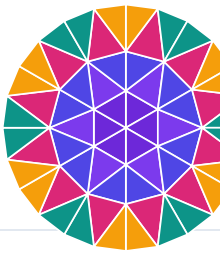
How cost is calculated

Every model has a price per million input tokens and per million output tokens. For a call with 2,000 input and 500 output tokens on a model that costs $0.15 and $0.60 per million: **2,000 × 0.15 / 1,000,000 + 500 × 0.60 / 1,000,000 = $0.0006**. kaleidoscope does this for you on every call.

kaleidoscope · DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR

PART I · GETTING STARTED · CHAPTER 03

Before you start



IN THIS CHAPTER

- What your project needs
- How to check your Flutter version

Requirement	Minimum	Why
Flutter	3.35	Uses current Flutter APIs
Dart	3.9	Comes with Flutter 3.35
Android	API 23 (Android 6.0)	Needed by the secure key storage
iOS	13.0	Needed by the secure key storage
An AI provider	Any	An API key, a local model, or an SDK you already use

Check your Flutter version

Open a terminal in your project folder and run:

```
flutter --version
```

If the version is older than 3.35, update with:

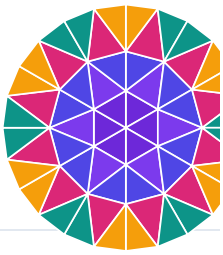
```
flutter upgrade
```

Keep API keys out of your app

Anything inside an app can be extracted by a determined person. For production apps, call AI providers through your own backend or a proxy that holds the key. The examples in this guide use a key directly only to keep them short. kaleidoscope works the same way either way.

kaleidoscope
DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR

Install in five minutes



IN THIS CHAPTER

- Adding the package
- Platform settings for Android and iOS
- Starting kaleidoscope in main()
- Opening the dashboard for the first time

1 Add the package

In your project folder, run:

```
flutter pub add kaleidoscope
```

Or add it by hand to `pubspec.yaml` and run `flutter pub get`:

pubspec.yaml

```
dependencies:
  kaleidoscope: ^1.0.0
```

2 Android: check the minimum SDK

Open `android/app/build.gradle.kts` (or `build.gradle`). If `minSdk` is lower than 23, set it to 23:

android/app/build.gradle.kts

```
android {
    defaultConfig {
        minSdk = 23
    }
}
```

3 iOS: check the platform version

Open `ios/Podfile` and make sure the platform line says 13.0 or higher:

ios/Podfile

```
platform :ios, '13.0'
```

4 Start kaleidoscope in main()

Open `lib/main.dart`. Add the import and the two marked lines:

lib/main.dart

```dart
import 'package:flutter/material.dart';
import 'package:kaleidoscope/kaleidoscope.dart';

Future<void> main() async {
  WidgetsFlutterBinding.ensureInitialized();   // add this
  await Kaleidoscope.init();                   // and this
  runApp(const MyApp());
}
```

That is the whole setup. With no arguments kaleidoscope uses safe defaults: prompts are not stored, records are kept for 90 days, and console logs appear only in debug builds.

5 Run your app

Start the app with `flutter run`. In the debug console you will see:

```
ℹ [kaleidoscope] ready (SecureSqliteUsageStore, prompts: metadataOnly)
```

What you should see

The line above means the encrypted database was created and kaleidoscope is listening. No AI calls are recorded yet, because we have not connected any. That is the next chapter.

6 Add a way to open the dashboard

Put this button anywhere in your app, for example on a settings or debug page:

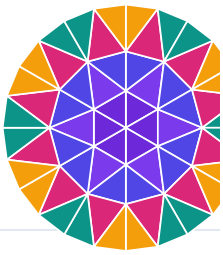
```dart
TextButton(
  onPressed: () => Kaleidoscope.openDashboard(context),
  child: const Text('AI usage'),
)
```

The dashboard opens only in debug builds unless you allow it in release builds (see Chapter 14).

kaleidoscope
DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR

PART II · CONNECTING YOUR AI · CHAPTER 05

Choose how to connect



IN THIS CHAPTER

- The five ways to connect AI calls
- Which one fits your code

kaleidoscope can see your AI calls in several ways. Pick the row that matches how your app talks to AI today. You can mix them in one app, and every method produces the same kind of record.

If your app…	Use	Chapter
calls an AI REST API with `package:http`, or uses an SDK that accepts an `http.Client`	`Kaleidoscope.httpClient()`	6, 9, 10
calls an AI REST API with **Dio**	`Kaleidoscope.dioInterceptor()`	7
uses an SDK that hides the network (Firebase AI, Google AI, LangChain…)	`Kaleidoscope.track(...)`	8
streams answers word by word through an SDK	`Kaleidoscope.trackStream(...)`	9
runs a model on the phone (Gemini Nano, TFLite, MediaPipe, llama.cpp)	`Kaleidoscope.instance.trackOnDevice(...)`	10
gets answers from your own backend, which calls the AI	`Kaleidoscope.record(...)`	11

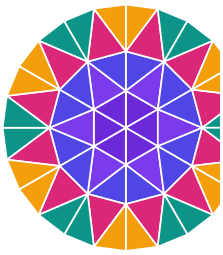
Not sure? Start with the HTTP client

The HTTP client and Dio interceptor need no extra code per call. They recognise OpenAI-compatible, Claude, Gemini, Ollama and Cohere requests automatically by their URL, on any host, proxy or gateway, including streaming responses.

APIs recognised automatically

API style	Recognised by	Examples
OpenAI-compatible	`/chat/completions`, `/responses`, `/embeddings`	OpenAI, Azure OpenAI, Groq, Mistral, DeepSeek, Together, OpenRouter, xAI, Perplexity, Fireworks, vLLM, LM Studio, LiteLLM
Anthropic Messages	`/v1/messages`	Claude models, including cache reads and writes
Gemini	`:generateContent`, `:streamGenerateContent`	Gemini API, Vertex AI, including thinking tokens
Ollama	`/api/chat`, `/api/generate`	Any local model (priced at zero)
Cohere	`/v2/chat`	Command models

Recipe: OpenAI with package:http



IN THIS CHAPTER

- Creating one tracked client for the whole app
- Sending a chat request
- What kaleidoscope records for it

1 Create one tracked client

Create it once and reuse it everywhere. It behaves exactly like a normal `http.Client`.

lib/ai/ai_client.dart

```dart
import 'package:kaleidoscope/kaleidoscope.dart';

final aiClient = Kaleidoscope.httpClient();
```

2 Call the API as usual

Nothing in the request changes. kaleidoscope reads the model, parameters and token usage on the way through.

lib/ai/openai.dart

```dart
import 'dart:convert';

Future<String> askOpenAI(String question) async {
  final response = await aiClient.post(
    Uri.parse('https://api.openai.com/v1/chat/completions'),
    headers: {
      'Authorization': 'Bearer $apiKey',
      'Content-Type': 'application/json',
    },
    body: jsonEncode({
      'model': 'gpt-4o-mini',
      'temperature': 0.3,
      'messages': [
        {'role': 'user', 'content': question},
      ],
    }),
  );
  final data = jsonDecode(response.body) as Map<String, dynamic>;
  return data['choices'][0]['message']['content'] as String;
}
```

3 Run it and watch the console

Each call prints one line in debug builds:

```
✔ [AI] openai/gpt-4o-mini chat • in 52 / out 31 • $0.000027 • 812ms
```

What you should see

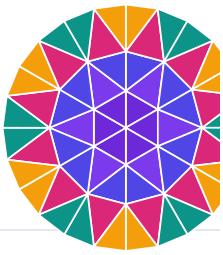
Open the dashboard: the call appears in the **Calls** tab with its model, tokens, cost, latency and the `temperature` parameter. Chapter 15 explains every part of the console line.

Using an OpenAI SDK package?

Most Dart SDKs accept a custom client. Pass the tracked one, for example `OpenAIClient(apiKey: key, client: Kaleidoscope.httpClient())` . Calls are then recorded with no other changes.

kaleidoscope
DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR

Recipe: Claude with Dio



IN THIS CHAPTER

- Adding the interceptor to Dio
- Calling the Claude Messages API
- How prompt caching shows up

1 Add the interceptor once

Add it where you create your `Dio` instance:

lib/ai/dio_client.dart

```dart
import 'package:dio/dio.dart';
import 'package:kaleidoscope/kaleidoscope.dart';

final dio = Dio()..interceptors.add(Kaleidoscope.dioInterceptor());
```

2 Call Claude

This is a normal Claude Messages request:

lib/ai/claude.dart

```dart
Future<String> askClaude(String question) async {
  final res = await dio.post(
    'https://api.anthropic.com/v1/messages',
    options: Options(headers: {
      'x-api-key': apiKey,
      'anthropic-version': '2023-06-01',
    }),
    data: {
      'model': 'claude-sonnet-5-5',
      'max_tokens': 1024,
      'messages': [
        {'role': 'user', 'content': question},
      ],
    },
  );
  return res.data['content'][0]['text'] as String;
}
```

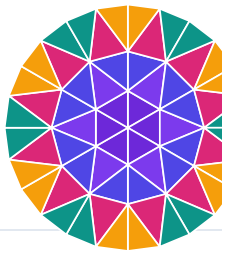
Prompt caching

When Claude reads part of your prompt from its cache, the record shows **cached input** and **cache write** tokens separately, and the cost uses the lower cache prices. The Overview tab shows the total money saved by caching.

Streaming works too

If you set `'stream': true` and use `ResponseType.stream`, kaleidoscope reads the stream as it passes through, records time to first token, and your code receives exactly the same bytes.

Recipe: any SDK with track()



IN THIS CHAPTER

- Wrapping an SDK call in Kaleidoscope.track
- Reading token usage from the SDK response
- Adding the prompt and parameters

Some SDKs do not let you pass an HTTP client. Wrap the call instead. `track` returns exactly what your call returns, and if the call throws, the error is recorded and thrown again unchanged.

This example uses the Gemini model from the `firebase_ai` package. The same pattern works for any SDK.

lib/ai/gemini.dart

```dart
final model = FirebaseAI.googleAI().generativeModel(model: 'gemini-2.5-flash');

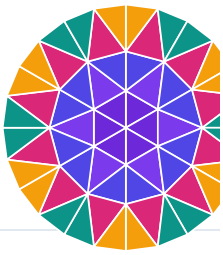
Future<String?> askGemini(String question) async {
  final response = await Kaleidoscope.track(
    provider: 'gemini',
    model: 'gemini-2.5-flash',
    prompt: question,                       // saved only if you allow prompts
    parameters: {'temperature': 0.4},       // shown in the call details
    call: () => model.generateContent([Content.text(question)]),
    usage: (r) => AITokenUsage(
      inputTokens: r.usageMetadata?.promptTokenCount ?? 0,
      outputTokens: r.usageMetadata?.candidatesTokenCount ?? 0,
    ),
    responseText: (r) => r.text,
  );
  return response.text;
}
```

Argument	Required	What it is for
`provider`, `model`	yes	Names used for pricing and in the dashboard
`call`	yes	Your SDK call, unchanged
`usage`	no	Reads token counts from the result. If you leave it out, tokens are estimated from the text
`prompt`, `responseText`	no	The text. Stored only when prompt storage is on
`parameters`	no	Any settings you want to see later, such as temperature
`context`, `customData`	no	Tags for this one call (see Chapter 12)

No usage data available?

Leave out `usage` and pass `prompt` and `responseText`. kaleidoscope estimates tokens from the text (about 4 characters per token) and marks the record as **estimated**, so you always know which numbers are exact.

Recipe: streaming answers



IN THIS CHAPTER

- Streams through the HTTP client or Dio
- Streams through an SDK with trackStream
- What time to first token tells you

Through the HTTP client or Dio

Nothing extra is needed. When a response is a stream (server-sent events or newline-delimited JSON), kaleidoscope copies the bytes as they pass, rebuilds the answer and the usage at the end, and measures time to first token. Your code receives the identical stream.

Get exact token counts from OpenAI-style streams

OpenAI-compatible APIs only include usage in a stream when asked. Turn this on and kaleidoscope adds the option for you: `KaleidoscopeConfig(injectStreamUsageOption: true)`. Without it, tokens are estimated and marked as such.

Through an SDK

Wrap the stream with `trackStream`. For each chunk, tell kaleidoscope the text and, if the SDK provides it, the usage:

lib/ai/gemini_stream.dart

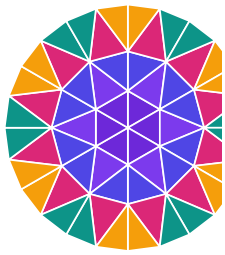
```dart
final stream = Kaleidoscope.trackStream<GenerateContentResponse>(
  provider: 'gemini',
  model: 'gemini-2.5-flash',
  prompt: question,
  call: () => model.generateContentStream([Content.text(question)]),
  onChunk: (chunk) => (
    text: chunk.text,
    usage: chunk.usageMetadata == null
        ? null
        : AITokenUsage(
            inputTokens: chunk.usageMetadata!.promptTokenCount ?? 0,
            outputTokens: chunk.usageMetadata!.candidatesTokenCount ?? 0,
          ),
  ),
);

await for (final chunk in stream) {
  setState(() => answer += chunk.text ?? '');
}
```

What you should see

The console line ends with `(ttft 140ms)`. In the dashboard, the **Trends** tab shows time to first token next to total latency. If users say the app "feels slow", look at TTFT first.

Recipe: local and on-device models



IN THIS CHAPTER

- Tracking a local Ollama model
- Tracking a model that runs on the phone
- Why these calls cost zero

Ollama on your computer

Ollama calls are recognised by the tracked HTTP client. From the Android emulator, your computer is reachable at `10.0.2.2`; from the iOS simulator use `localhost`.

lib/ai/ollama.dart

```
final res = await aiClient.post(
  Uri.parse('http://10.0.2.2:11434/api/chat'),
  body: jsonEncode({
    'model': 'llama3.2',
    'stream': false,
    'messages': [
      {'role': 'user', 'content': question},
    ],
  }),
);
```

Models on the device

For Gemini Nano, TensorFlow Lite, ML Kit, MediaPipe or llama.cpp, wrap the call with `trackOnDevice`. If you do not know the token counts, give the input and output text and they are estimated.

lib/ai/on_device.dart

```
final reply = await Kaleidoscope.instance.trackOnDevice(
  model: 'gemini-nano',
  runtime: 'android-aicore',
  inputText: question,
  call: () ⇒ nano.generate(question),
  outputText: (result) ⇒ result,
);
```

Free, but not invisible

Local and on-device models are priced at zero, but they still take time and battery. Tracking them shows how often they run, how long they take and which screens use them, so you can compare them fairly with cloud models.

Recipe: your backend and your own gateway

IN THIS CHAPTER

- Recording calls your server makes
- Tracking your company AI gateway
- Adding support for any other API

When your server calls the AI

If your app sends a request to your own backend and the backend calls the AI, have the backend return the usage numbers, then record them in the app. The call is tagged with the current screen like any other.

```
final result = await myApi.askAssistant(question);

Kaleidoscope.record(
  provider: 'openai',
  model: result.model,                    // e.g. 'gpt-4o'
  inputTokens: result.inputTokens,
  outputTokens: result.outputTokens,
  latencyMs: result.latencyMs,
);
```

Your own gateway

If your gateway speaks one of the recognised API styles, it is detected by its URL path automatically. If it uses its own format, tell kaleidoscope to inspect its host; a generic reader finds the common usage fields (`usage` , `prompt_tokens` , `input_tokens` and similar):

```
await Kaleidoscope.init(
  config: KaleidoscopeConfig(
    additionalHostsToTrack: ['ai.mycompany.com'],
  ),
);
```

Reading usage yourself

Ready-made readers turn any provider response into an `AITokenUsage` : `UsageExtractors.openAI(json)` , `.anthropic(json)` , `.gemini(json)` , `.ollama(json)` , `.cohere(json)` and `.auto(json)` , which detects the format. They are handy inside `track(usage: ...)` .

Advanced: a custom adapter

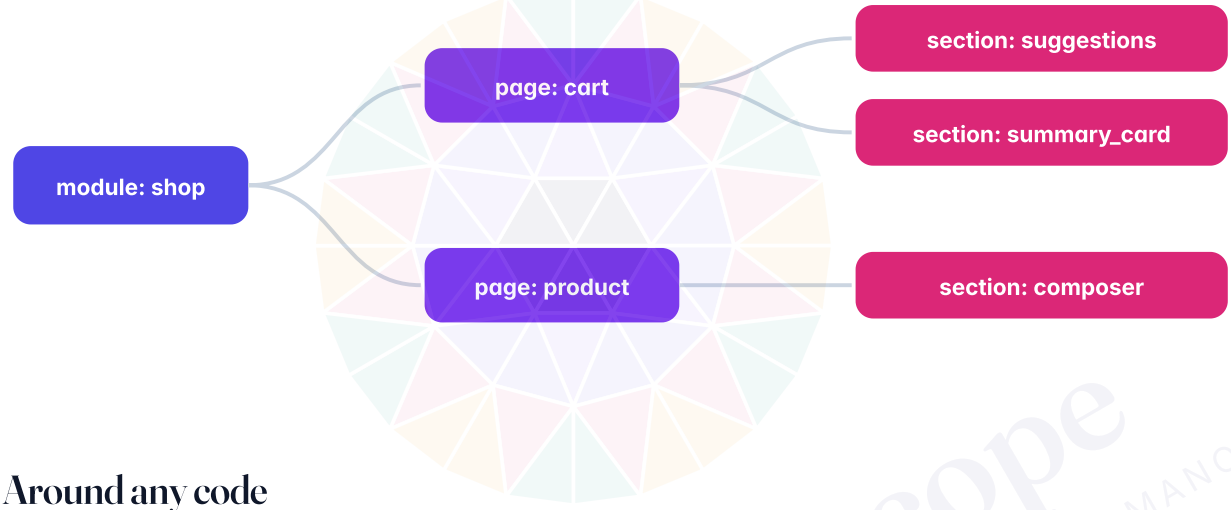
For a completely different API, implement `AIProviderAdapter` (how to recognise the request, read the model and read the usage) and register it with `Kaleidoscope.instance.registerAdapter(MyAdapter())` .

Tag calls by module, page and section

IN THIS CHAPTER

- Why tags matter
- Three ways to tag calls
- Extra fields such as user, feature and your own data

Knowing that you spent $40 on AI this month is useful. Knowing that $31 of it came from the **suggestions** section of the **cart** page is what lets you act. Tags form a simple tree:



1. Around any code

Everything inside the function is tagged, including calls made several `await`s later and calls captured by the HTTP client or Dio.

```
final reply = await Kaleidoscope.scope(
  module: 'shop',
  page: 'cart',
  section: 'suggestions',
  () ⇒ aiClient.post(url, body: body),
);
```

2. Automatically from your routes

Add the observer and every call is tagged with the current route. A route named `/shop/cart` becomes module `shop` and page `/shop/cart`.

```
MaterialApp(
  navigatorObservers: [Kaleidoscope.navigatorObserver()],
  // ...
)
```

3. In the widget tree

Wrap a screen once, then tag sections inside it:

```
// around the page
AIUsageScope(module: 'support', page: 'chat', child: const ChatPage());

// inside ChatPage
AIUsageScope.run(context, () ⇒ askOpenAI(text), section: 'composer');
```

Extra fields

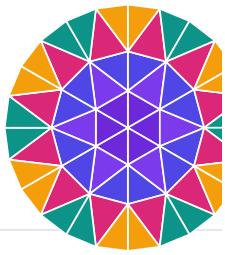
Scopes also accept `feature`, `userId`, `conversationId`, `tags` and a `custom` map. Inner scopes merge with outer ones, the most specific value wins. To add the same tags to every call, such as the app flavour, use `defaultContext` in `init`.

```
await Kaleidoscope.init(
  defaultContext: const AIUsageContext(custom: {'plan': 'pro', 'flavor': 'prod'}),
);

// later: cost per plan
final byPlan = await Kaleidoscope.analytics
    .breakdown(UsageDimension.module, customKey: 'plan');
```

kaleidoscope
DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR

A tour of the dashboard

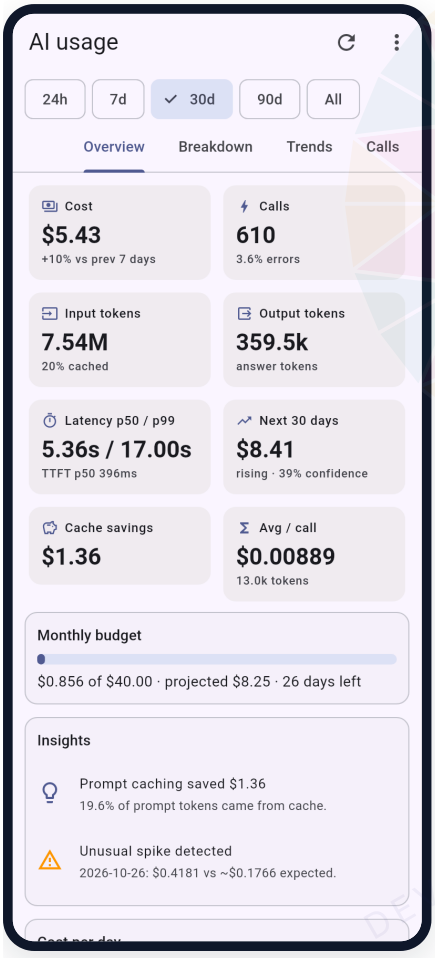


IN THIS CHAPTER

- The four tabs and what each card means
- Searching the call log
- Choosing a time range

Open it with `Kaleidoscope.openDashboard(context)`, or push `AIAnalyticsScreen()` like any other screen. It updates live as calls happen and follows your app's light or dark theme. The chips at the top switch between the last 24 hours, 7, 30 or 90 days, or all data.

Overview



Overview tab
Key numbers, budget and insights

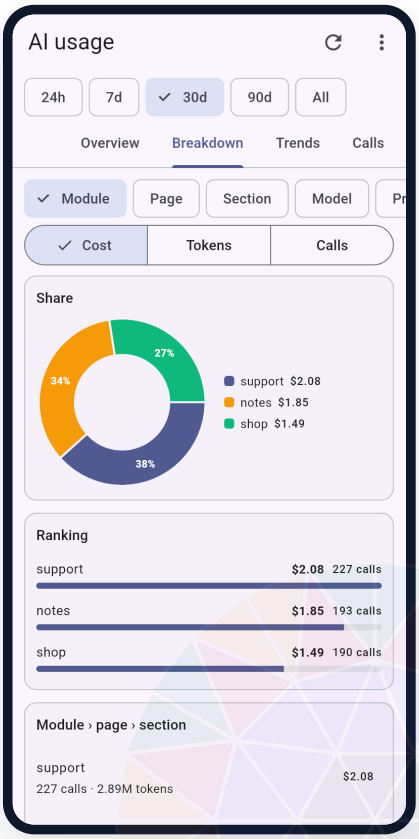
Card	Meaning
Cost	Money spent in the range, and the change against the previous period
Calls	Number of AI calls and the share that failed
Input / output tokens	Tokens sent and received; share of input served from cache
Latency p50 / p99	Typical and worst-case duration, plus time to first token
Next 30 days	Forecast spend, its trend and how confident it is
Cache savings	Money saved by provider prompt caching
Monthly budget	Spend so far, projection to month end, days left
Insights	Plain-language findings, such as spikes, errors and savings

Below the cards you will find daily cost with a forecast band, and cost by model and by module.

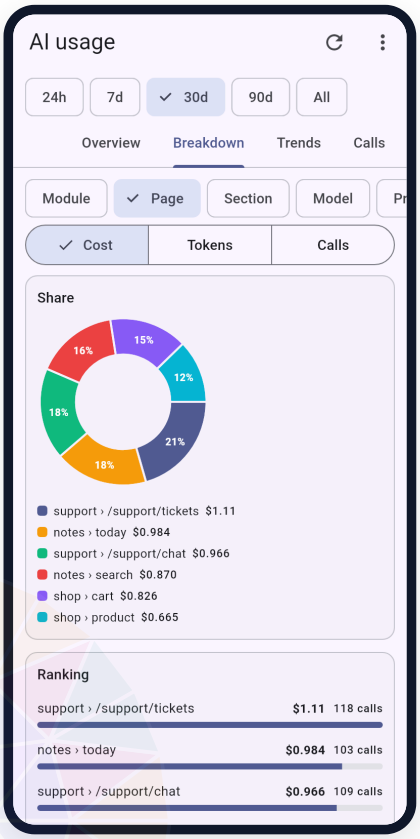
Breakdown

Choose what to group by (module, page, section, model, provider or feature) and what to measure (cost, tokens or calls). The donut shows each share, the ranking lists them in order, and the **module › page › section** tree lets you

drill down to the exact part of a screen.



Breakdown by page
Share, ranking and table

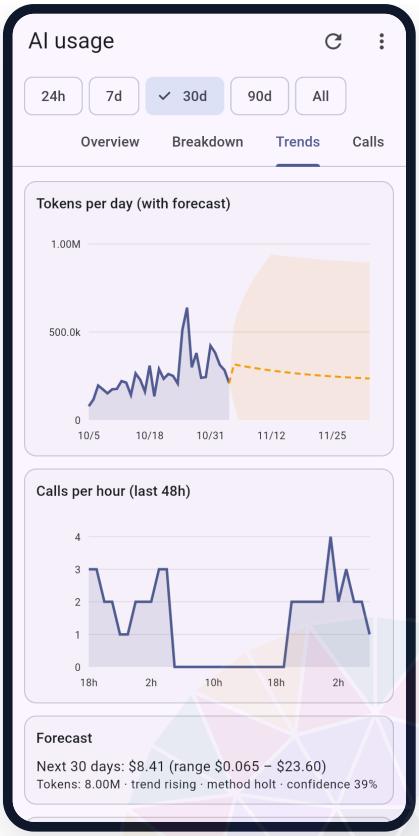


Module › page › section
Expandable tree

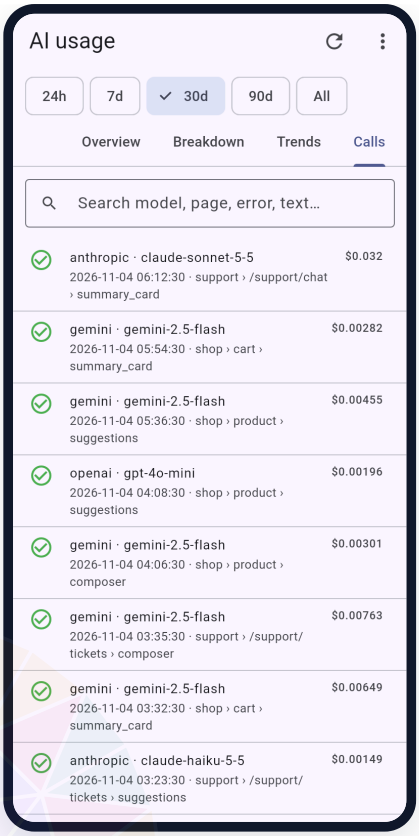
Trends

Tokens per day with a forecast, calls per hour for the last two days, latency percentiles (p50 to p99) with time to first token and tokens per second, a weekday by hour heatmap that shows when your users use AI the most, and a list of unusual days.

kaleidoscope
DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR



Trends
Forecast, hourly calls, latency, heatmap, anomalies

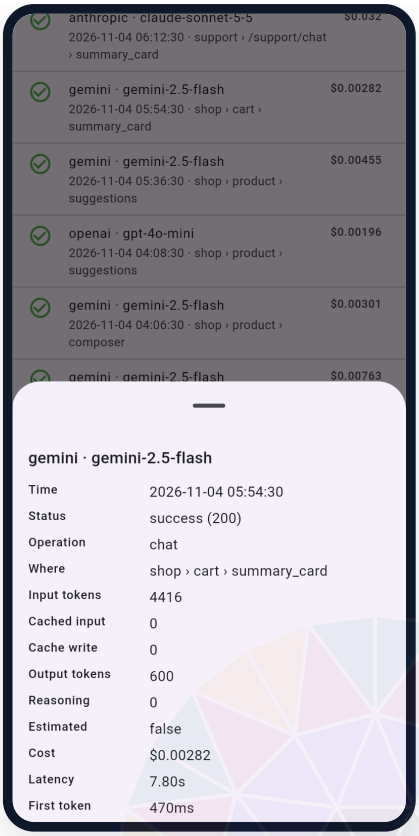


Calls
Searchable log of every call

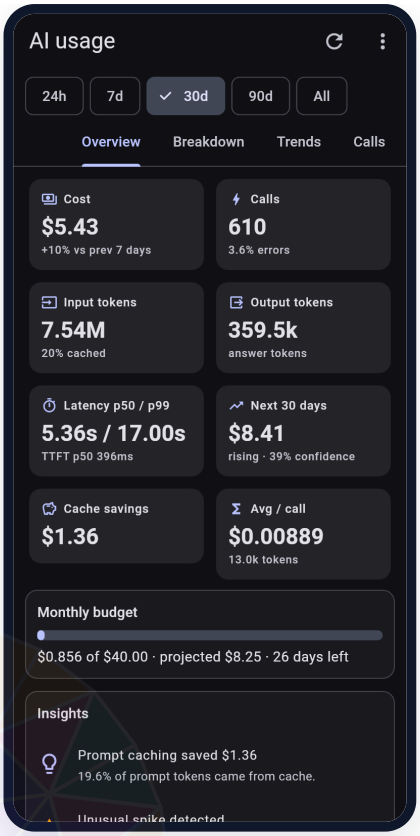
Calls and call details

The **Calls** tab lists every call, newest first, with its model, where it came from, tokens, latency and cost. Search by model, page, error text or anything else. Tap a call to see everything that was recorded: status, every token type, cost, latency and time to first token, all parameters, your tags and custom data, and the prompt and answer if you allowed them to be stored.

kaleidoscope
DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR



Call details
Every recorded field



Dark theme
Follows your app automatically

kaleidoscope
DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR

Using the dashboard in release builds

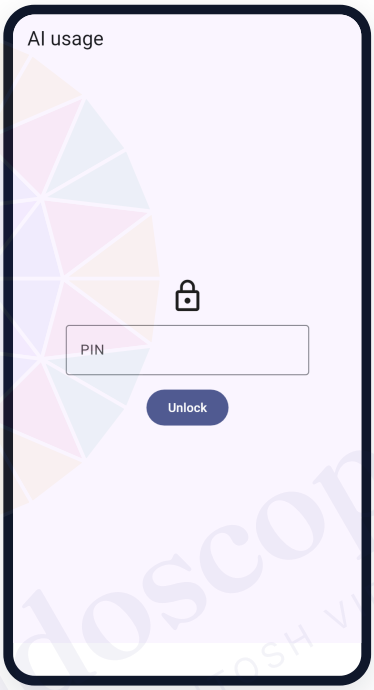
IN THIS CHAPTER

- Why it is hidden by default
- Showing it to admins with a PIN
- Exporting and deleting data from the menu

By default the dashboard opens only in debug builds, so your users never see it by accident. To show it in a release build, for example to an admin or a tester, allow it and protect it with a PIN:

```
Navigator.of(context).push(MaterialPageRoute(
  builder: (_) => AIAnalyticsScreen(
    allowInRelease: true,
    accessPin: '2580',
    title: 'AI usage',
    onExport: (format, content) {
      // save or share the file
      Share.share(content);
    },
  ),
));
```

The shortcut `Kaleidoscope.openDashboard(context, accessPin: '2580', allowInRelease: true)` does the same.



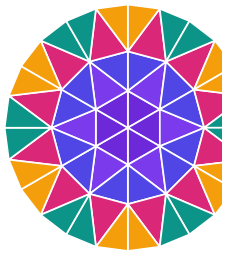
PIN screen
Shown before the dashboard opens

The menu

Action	What it does
Export JSON / CSV	The report and the raw records. Copied to the clipboard, or given to your `onExport` callback so you can save or share a file
Export HTML report	A single, self-contained page with charts you can open in any browser or attach to an email
Delete all data	Wipes the database after you confirm. Hide this option with `allowClear: false`

kaleidoscope DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR

Debug console and floating overlay



IN THIS CHAPTER

- Reading a console line
- Choosing how much is printed
- Adding the live overlay

While you develop, every call is printed to the debug console in VS Code, Android Studio or the terminal. Lines are colour-coded and also appear in Flutter DevTools under the `Kaleidoscope` logger. Nothing is printed in release builds.

Anatomy of a line

```
✔[1] [AI] anthropic/claude-haiku-5-5 chat (stream)[2] @ shop › cart › suggestions[3] • in 22 (+1200 cached) / out 18[4] • $0.000104[5] • 905ms (ttft 140ms)[6]
```

#	Part	Meaning
1	✔ / ✖	Success or failure. Failed lines end with the status code and error message
2	Provider, model, operation	Who answered and what kind of call it was; "stream" for streamed answers
3	Tags	Module › page › section the call came from
4	Tokens	Input (with cached tokens in brackets) and output
5	Cost	In your price table currency
6	Speed	Total latency and, for streams, time to first token

How much to print

```dart
await Kaleidoscope.init(debugLog: AIDebugLogLevel.verbose);
```

Level	Prints
`off`	Nothing
`errors`	Failed calls only
`summary` (default)	One line per call
`verbose`	Also parameters and custom data. Prompt text only if you set `showPromptText: true` in `AIDebugLogConfig`, and always redacted

Kaleidoscope DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR

The floating overlay

A small bubble shows the session cost, number of calls, tokens and the last latency, and flashes on every call. Tap it to see the latest calls; long-press to open the dashboard; drag it anywhere. It is removed automatically in profile and release builds, so you can leave the code in place.

lib/main.dart

```dart
final navKey = GlobalKey<NavigatorState>();

MaterialApp(
  navigatorKey: navKey,
  builder: (context, child) =>
      AIUsageDebugOverlay(navigatorKey: navKey, child: child!),
);
```

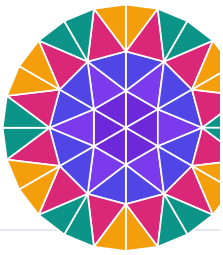


Collapsed
Session totals, flashes on each call

Expanded
Latest calls with cost, speed and place

kaleidoscope
DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR

Privacy and security



IN THIS CHAPTER

- Choosing whether prompts are stored
- Removing personal data automatically
- How the data is encrypted
- Retention: deleting old data

Prompt storage: you decide

By default kaleidoscope never stores the text of prompts or answers, only the numbers and tags. You can choose one of three modes:

Mode	How to turn it on	What is stored
Metadata only (default)	nothing to do	Tokens, cost, model, parameters, timing and tags. No text
Redacted	`promptStorage: PromptStorageMode.redacted`	Text, with emails, phone numbers, card numbers, API keys, bearer tokens and IP addresses replaced
Full	`storePrompts: true`	The complete text, still encrypted on the device

```
await Kaleidoscope.init(
  promptStorage: PromptStorageMode.redacted,
  keepPromptTextFor: const Duration(days: 7), // remove text after 7 days,
);                                            // keep the numbers
```

Your own redaction rules

Add patterns for anything private in your app, such as order numbers or customer IDs. Each rule is a name, a regular expression and a replacement:

```
await Kaleidoscope.init(
  config: KaleidoscopeConfig(
    promptStorage: PromptStorageMode.redacted,
    redactors: [
      ...Redactors.defaults,
      const Redactor('order', r'#\d{3,}', '[ORDER]'),
    ],
  ),
);
```

Encryption

Records are stored in a SQLCipher database encrypted with AES-256. On first launch a random 256-bit key is created and kept in the iOS Keychain or the Android Keystore; it never leaves secure storage. If secure storage is not available on a device, kaleidoscope keeps records in memory for that session instead, and your app keeps working.

Retention

```
await Kaleidoscope.init(
  keepFor: const Duration(days: 90), // delete records older than 90 days
  maxRecords: 50000,                 // and never keep more than 50,000
);
```

Cleanup runs at start-up and every 6 hours. You can also run it at any moment with `Kaleidoscope.instance.runRetention()`, delete everything with `Kaleidoscope.instance.clearAll()`, or change or drop single records before they are saved with `beforeSave`:

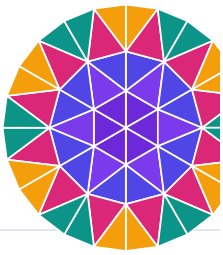
```
KaleidoscopeConfig(
  beforeSave: (record) ⇒
      record.context.module == 'health' ? record.copyWith(clearText: true) : record,
)
```

Nothing leaves the device unless you add a sink

kaleidoscope has no servers and sends nothing anywhere by default. Data is shared only if you add a sink (Chapter 20), and sinks send numbers only unless you turn on `includeText`. Even then, text is scrubbed of personal data first.

kaleidoscope
DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR

Costs, prices and budgets



IN THIS CHAPTER

- How prices are looked up
- Updating prices in the app or from a file you host
- Estimating cost before you send
- Setting a monthly budget

kaleidoscope ships with prices for more than 30 popular models, including prompt-cache discounts. Providers change prices, so you can update them at any time without releasing a new app version.

Change a price in code

```
// Saved on the device and reused on the next launch
Kaleidoscope.pricing.setPrice(
  'gpt-4o*',
  const ModelPrice(inputPerMillion: 2.5, outputPerMillion: 10, cachedInputPerMillion: 1.25),
);

// Your own model, in your own currency
Kaleidoscope.pricing.setPrice(
  'acme-support-*',
  const ModelPrice(inputPerMillion: 40, outputPerMillion: 120, currency: 'INR'),
);
```

Or keep prices in a file you control

Host a small JSON file (on your website, a CDN or cloud storage). kaleidoscope downloads it at start-up and refreshes it every 24 hours:

```
await Kaleidoscope.init(
  remotePricingUrl: Uri.parse('https://api.example.com/ai-prices.json'),
);
```

ai-prices.json

```
{
  "version": "2026-11",
  "currency": "USD",
  "models": {
    "gpt-4o-mini*":     { "input": 0.15, "output": 0.6, "cachedInput": 0.075 },
    "claude-sonnet-5*": { "input": 2, "output": 10, "cachedInput": 0.2, "cacheWrite": 2.5 },
    "ollama:*":         { "input": 0, "output": 0 }
  }
}
```

Key style	Matches
`gpt-4o`	Exactly that model
`gpt-4o*` or `*-mini`	Prefix or suffix; the longest matching key wins
`ollama:*`	Every model from one provider

Models with no known price are shown as **unpriced** and listed in Insights, so you know which price to add.

Estimate before sending

```
final e = Kaleidoscope.estimate(
  model: 'gpt-4o',
  prompt: longText,
  expectedOutputTokens: 500,
);
if (e.cost.total > 0.05) {
  // ask the user to confirm, or choose a cheaper model
}
```

A monthly budget

```
await Kaleidoscope.init(monthlyBudget: 40);

final status = await Kaleidoscope.analytics.budget(40);
print('${status.spent} spent, ${status.projected} projected, on track: ${status.onTrack}');
```

With a budget set, the Overview tab shows a progress bar and month-end projection, and Insights warns you when the projection goes over budget.

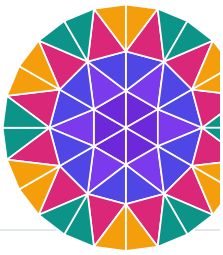
Prices are a starting point

The built-in prices were correct when this version was released, but providers change them. Check the prices for the models you use, and use a remote price file so you can correct them without an app update.

kaleidoscope
DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR

PART V · GOING FURTHER · CHAPTER 18

The analytics API



IN THIS CHAPTER

- Getting the same numbers the dashboard shows
- Filtering by time, model or place
- A worked example: cost this week

Everything in the dashboard is available as data, so you can build your own reports, send alerts or show usage to your users.

```
final a = Kaleidoscope.analytics;

await a.summary(from: DateTime.now().subtract(const Duration(days: 7)));
await a.breakdown(UsageDimension.model);      // also provider, module, page, section, feature...
await a.moduleTree();                         // module › page › section
await a.timeSeries(TimeBucket.day);           // hour, day, week, month
await a.latency();                            // p50, p90, p95, p99, TTFT, tokens per second
await a.forecast(metric: 'cost', days: 30);   // value with a 95% range and trend
await a.anomalies();                          // unusual days
await a.budget(40);                           // spent, projected, daily allowance
await a.compare(const Duration(days: 7));     // this week vs last week
await a.insights();                           // plain-language findings
final report = await a.report();              // everything at once
```

Every method accepts `from`, `to` and a `filter`. A filter can limit results to a provider, model, module, page, section, status or a search text:

```
final shopOnly = UsageQuery(modules: {'shop'});
final week = await a.summary(
  from: DateTime.now().subtract(const Duration(days: 7)),
  filter: shopOnly,
);
```

Worked example: a "cost this week" tile

```
class WeeklyCostTile extends StatelessWidget {
  const WeeklyCostTile({super.key});

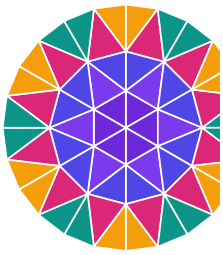
  @override
  Widget build(BuildContext context) {
    return FutureBuilder(
      future: Kaleidoscope.analytics.summary(
        from: DateTime.now().subtract(const Duration(days: 7)),
      ),
      builder: (context, snap) {
        if (!snap.hasData) return const CircularProgressIndicator();
        final t = snap.data!;
        return AIStatCard(
          label: 'AI cost this week',
          value: AIFormat.money(t.cost),
          caption: '${t.calls} calls',
        );
      },
    );
  }
}
```

Analysis	How it works
Forecast	Trend smoothing on complete days (Holt with a damped trend), with a 95% range. Today is excluded because it is still in progress
Anomalies	Days more than 2.5 standard deviations away from the previous two weeks
Budget	This month's spend plus the forecast for the remaining days
Insights	Top spenders, error rate, cache savings, unpriced models, weekly change, spikes, slow calls and budget risk

kaleidoscope
DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR

PART V · GOING FURTHER · CHAPTER 19

Build your own screens with chart widgets



IN THIS CHAPTER

- The ready-made chart widgets
- Feeding them with a report

The charts used by the dashboard are available as separate widgets. They take their colours from your app's theme, so they fit any design.

```
final report = await Kaleidoscope.analytics.report(
  from: DateTime.now().subtract(const Duration(days: 30)),
);

Column(children: [
  AIStatCard(label: 'Cost this month', value: AIFormat.money(report.totals.cost)),
  AIUsageLineChart(points: report.daily, forecast: report.costForecast),
  AIUsagePieChart(items: report.byModel),
  AIUsageBarList(items: report.byPage),
  AIUsageHeatmap(grid: report.heatmap),
]);
```

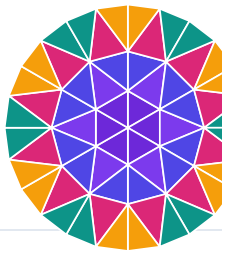
Widget	Shows
`AIStatCard`	One number with a label, caption and icon
`AIUsageLineChart`	A daily series, with an optional forecast line and range band
`AIUsagePieChart`	Shares of a breakdown as a donut with a legend
`AIUsageBarList`	A ranked list with bars, for pages, models or modules
`AIUsageBarChart`	Vertical bars from values and labels
`AIUsageHeatmap`	Calls by weekday and hour

Formatting helpers

`AIFormat` formats money with currency symbols, token counts (1.2k, 3.4M), durations and percentages the same way the dashboard does.

kaleidoscope DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR

Export and integrations



IN THIS CHAPTER

- Exporting data as JSON, CSV or HTML
- Sending records to your server or analytics tools
- Running the sample server

Export

```
final html = await Kaleidoscope.exportHtml();   // a report with charts, no scripts
final csv  = await Kaleidoscope.exportCsv();    // one row per call, for spreadsheets
final json = await Kaleidoscope.exportJson();   // everything, for your own tools
```

Sinks: send copies of records elsewhere

Sinks are optional. Add only the ones you need:

```
Kaleidoscope.addSink(ServerSyncSink(
  endpoint: Uri.parse('https://api.example.com/ai-usage'),
  headers: {'Authorization': 'Bearer <your-token>'},
));
Kaleidoscope.addSink(PostHogSink(apiKey: 'phc_...'));
Kaleidoscope.addSink(MixpanelSink(projectToken: '...'));
Kaleidoscope.addSink(OpenTelemetrySink(
  tracesEndpoint: Uri.parse('https://otel.example.com/v1/traces'),
));
Kaleidoscope.addSink(OpenTelemetrySink.langfuse(publicKey: 'pk-lf-...', secretKey: 'sk-lf-...'));
Kaleidoscope.addSink(CallbackSink('firebase', (batch) async {
  // write the batch anywhere you like
}));
```

Behaviour	Details
Batching	Records are sent in batches, when a batch fills up or on a timer
Offline	Unsent records wait in the encrypted database and survive app restarts
Retries	Failed sends are retried with growing delays, up to 15 minutes apart
Privacy	Numbers and tags only, unless you set `includeText: true` (text is scrubbed first)
OpenTelemetry	One span per call using the GenAI conventions (`gen_ai.*`), for Grafana, Honeycomb, Datadog or Langfuse

The sample server

The package includes a tiny receiver for `ServerSyncSink` with no dependencies. It removes duplicates and offers a summary endpoint:

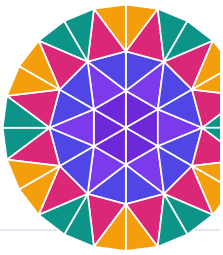
```
cd example/server
dart run server.dart          # listens on http://localhost:8080
# POST /ai-usage   receives batches
# GET  /summary    totals by model and module
```

Set the `AI_USAGE_TOKEN` environment variable to require a bearer token, and `PORT` to change the port.

kaleidoscope
DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR

PART V · GOING FURTHER · CHAPTER 21

Provider fallback



IN THIS CHAPTER

- Trying a second provider when the first fails
- Skipping a provider that keeps failing

This is optional. If your app can use more than one model, the router tries them in order and moves on when one fails or times out. Every attempt is recorded, so you can see how often fallback happens.

```
final router = AIFallbackRouter(
  failureThreshold: 3,                    // after 3 failures in a row...
  cooldown: const Duration(minutes: 1),   // ...skip that provider for a minute
);

final answer = await router.run([
  AIRoute(provider: 'openai', model: 'gpt-4o-mini',
      call: () => askOpenAI(question), timeout: const Duration(seconds: 20)),
  AIRoute(provider: 'anthropic', model: 'claude-haiku-5-5',
      call: () => askClaude(question)),
]);
```

Avoid counting calls twice

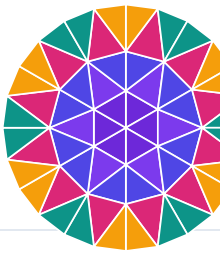
If the calls inside the routes already go through the tracked HTTP client or Dio, create the router with `trackAttempts: false`. The HTTP layer records them already.

If every route fails, `run` throws `AIAllRoutesFailedException` with the error from each route.

kaleidoscope
DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR

PART V · GOING FURTHER · CHAPTER 22

Testing your app



IN THIS CHAPTER

- Using kaleidoscope in widget and unit tests

In tests, use the in-memory store so nothing is written to disk and no secure storage is needed, and turn console output off:

test/ai_usage_test.dart

```
setUp(() async {
  await KaleidoscopeTracker.instance.initialize(
    KaleidoscopeConfig(
      store: InMemoryUsageStore(),
      debugLog: AIDebugLogConfig.off,
    ),
  );
});

test('the cart suggestions call is tracked', () async {
  await Kaleidoscope.scope(module: 'shop', page: 'cart', () => cart.loadSuggestions());
  final records = await Kaleidoscope.records();
  expect(records.single.context.module, 'shop');
});
```

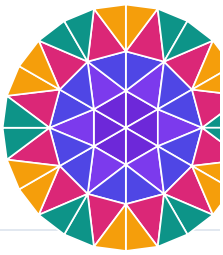
Try the example app

The `example` folder is a complete app that runs without any API keys. It shows every connection style, tagging, streaming, on-device tracking, fallback, sample data and the dashboard. Run it with `cd example && flutter run`.

kaleidoscope
DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR

PART VI · REFERENCE · CHAPTER 23

Troubleshooting



IN THIS CHAPTER

- Common problems and how to fix them

Problem	Likely cause	Fix
No calls appear	The call does not go through the tracked client	Use `Kaleidoscope.httpClient()` or the Dio interceptor, or wrap the call with `track`
Still no calls with the HTTP client	The API has an unusual URL	Add the host to `additionalHostsToTrack`, and check it is not in `ignoredHosts`
Calls made before init are missing	They were queued	Calls made during start-up are buffered (up to 1,000) and saved once kaleidoscope is ready
Tokens marked "estimated"	The provider sent no usage	Common for OpenAI-style streams: set `injectStreamUsageOption: true`
Cost shows $0 or "unpriced"	No price matched the model name	Add one with `Kaleidoscope.pricing.setPrice(...)` or in your remote price file
Calls have no module or page	No scope or route name	Add `navigatorObserver()`, name your routes, or use `scope`
Calls counted twice	Both the HTTP layer and `track` or the router record them	Use one method per call, or `trackAttempts: false` on the router
Android build error about minSdk	minSdk below 23	Set `minSdk = 23` in `android/app/build.gradle.kts`
Conflict with the sqflite package	Two SQLite libraries in one app	See the sqflite_sqlcipher README on using a single SQLite library
Dashboard does not open in release	Hidden by default	Pass `allowInRelease: true`, ideally with an `accessPin`
Overlay not visible	Release or profile build, or no navigator key	The overlay shows only in debug builds; pass `navigatorKey` so long-press can open the dashboard

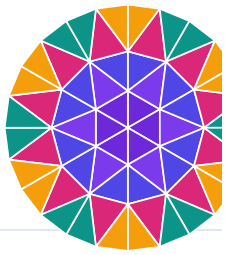
See kaleidoscope's own errors

kaleidoscope never throws into your app. To see its internal problems, pass `onError: (error, stack) ⇒ ...` in `KaleidoscopeConfig`, for example to forward them to Crashlytics.

kaleidoscope DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR

PART VI · REFERENCE · CHAPTER 24

Frequently asked questions



IN THIS CHAPTER

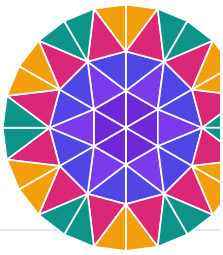
- Short answers to the questions people ask most

Question	Answer
Can kaleidoscope break my AI calls?	No. Every internal step is guarded. Requests and responses pass through byte for byte, and errors from your call are recorded and rethrown unchanged. The test suite covers broken storage, broken parsers and malformed responses.
Does it slow my app down?	Hardly. Records are written in the background off the request path, and analytics run only when you ask for them.
Does any data leave the device?	Only if you add a sink. There is no kaleidoscope server.
Which AI models are supported?	All of them. Common APIs are recognised automatically, and anything else works with `track`, `record` or a custom adapter.
Is it free?	Yes. kaleidoscope is open source under the BSD 3-Clause licence, free for personal and commercial use.
Does it work on web or desktop?	Version 1.0 supports iOS and Android, where encrypted storage is available.
Can I store prompts for debugging only?	Yes. For example, use `storePrompts: kDebugMode`, or redacted mode with a short `keepPromptTextFor`.
Can I show users their own usage?	Yes. Use the analytics API with a `userId` tag and the chart widgets, or the dashboard in release mode.
What happens when secure storage is unavailable?	kaleidoscope keeps records in memory for that session, and your app keeps working normally.
How accurate are costs?	Exact token counts from the provider multiplied by your price table. Estimated records are clearly marked.

kaleidoscope
DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR

PART VI · REFERENCE · CHAPTER 25

Reference



IN THIS CHAPTER

- Every configuration option
- The main API at a glance

The shortcuts in `Kaleidoscope.init(...)` cover the common settings. For everything else, pass `config: KaleidoscopeConfig(...)`.

Option	Default	Purpose
`enabled`	true	Master switch
`promptStorage`	metadataOnly	metadataOnly, redacted or full
`maxStoredTextLength`	20000	Longer text is truncated
`redactors`	built-in set	Patterns removed in redacted mode
`retention`	90 days, 50,000	`RetentionPolicy(maxAge, maxRecords, textMaxAge, cleanupInterval)`
`debugLog`	summary	`AIDebugLogConfig(level, useColors, showPromptText)`
`store`	encrypted SQLite	Any `UsageStore`, such as `InMemoryUsageStore`
`databaseName`	kaleidoscope.db	File name of the database
`pricing`	built-in	Your own `PricingTable`
`remotePricingUrl`	none	Price file refreshed every `pricingRefreshInterval` (24 h)
`captureParameters`	true	Record request parameters such as temperature
`estimateMissingTokens`	true	Estimate tokens when the provider reports none
`injectStreamUsageOption`	false	Ask OpenAI-style streams to include usage
`additionalHostsToTrack`	none	Extra hosts for the generic reader
`ignoredHosts`	none	Hosts never inspected
`beforeSave`	none	Change or drop records before saving
`onError`	none	Receive kaleidoscope's own internal errors
`defaultContext`	none	Tags added to every record
`monthlyBudget`	none	Budget card, projection and warnings

Kaleidoscope DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR

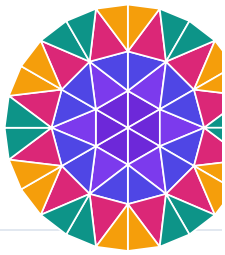
API at a glance

Call	What it does
`Kaleidoscope.init(...)`	Start kaleidoscope (once, in main)
`Kaleidoscope.httpClient()`	A tracked `http.Client`
`Kaleidoscope.dioInterceptor()`	A tracked Dio interceptor
`Kaleidoscope.track(...)`	Wrap any AI call
`Kaleidoscope.trackStream(...)`	Wrap a streaming AI call
`Kaleidoscope.instance.trackOnDevice(...)`	Wrap an on-device model call
`Kaleidoscope.record(...)`	Record a call measured elsewhere
`Kaleidoscope.scope(...)`	Tag calls with module, page, section and more
`Kaleidoscope.navigatorObserver()`	Tag calls with the current route
`Kaleidoscope.openDashboard(context)`	Open the analytics screen
`Kaleidoscope.analytics`	Summaries, breakdowns, trends, forecasts, budget
`Kaleidoscope.pricing`	Read and update prices
`Kaleidoscope.estimate(...)`	Estimate tokens and cost before sending
`Kaleidoscope.addSink(...)`	Send records to a server or tool
`Kaleidoscope.records(...)`	Read raw records
`Kaleidoscope.exportJson / exportCsv / exportHtml`	Export data
`Kaleidoscope.instance.clearAll()`	Delete all records
`AIUsageScope`, `AIUsageDebugOverlay`, `AIAnalyticsScreen`	Widgets for tagging, the overlay and the dashboard

kaleidoscope
DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR

PART VI · REFERENCE · CHAPTER 26

Checklist before you ship



IN THIS CHAPTER

- The settings to review before releasing your app

1. **API keys are not in the app**

   Production AI calls go through your backend or a proxy that holds the keys.

2. **Prompt storage is what you intend**

   Metadata only, or redacted with your own rules. Avoid full text in release builds unless users agreed to it.

3. **Retention fits your privacy policy**

   `keepFor`, `maxRecords` and `keepPromptTextFor` match what you tell users.

4. **The dashboard is hidden or protected**

   It is hidden in release by default. If you enable it, use `accessPin`.

5. **Sinks send only what you expect**

   Check endpoints and keep `includeText` off unless you need it.

6. **Prices are current**

   Check the models you use, and consider a remote price file.

7. **Your privacy policy mentions it**

   If records leave the device through a sink, describe that data in your privacy policy and in the app store privacy forms.

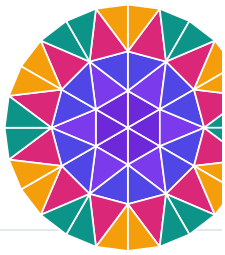
You are ready

The console logs and the overlay remove themselves in release builds, so you can leave all debugging code in place. Ship with confidence, then use the dashboard and analytics to make your AI features faster and cheaper.

kaleidoscope
DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR

PART VI · REFERENCE · CHAPTER 27

Legal notices and disclaimer



IN THIS CHAPTER

- Why kaleidoscope is an experimental project
- Who makes kaleidoscope and who does not
- Why every number must be cross-checked
- Where the full Terms and Conditions live

kaleidoscope is an independent open source project developed by Sanket Santosh Vidya Manohar. Please read this short summary, and the full Terms and Conditions, before you rely on the package in a product.

A free, experimental project

kaleidoscope is a free, open source and experimental project, shared with the community for learning and evaluation. It is not a commercial product or a paid service. The version number describes the stability of the programming interface only; it does not mean the package is production-ready, audited or guaranteed. You use it entirely at your own risk, and if it does not suit you, the remedy is simply to stop using it.

No affiliation or endorsement

kaleidoscope is not affiliated with, sponsored by, endorsed by or approved by OpenAI, Anthropic, Google, Microsoft, Meta, Mistral AI, Cohere, Ollama, Groq, xAI, DeepSeek, Amazon, Apple, the Flutter or Dart teams, or any other company or AI model provider. It simply reads the requests and responses that your own app already makes.

Trademarks and brand names

All product names, model names, logos and brands mentioned in this guide, on the website and in the code are the property of their respective owners. They are used only to describe compatibility. Their use does not imply any relationship, partnership or endorsement.

Figures can be wrong

Token counts, costs, forecasts, latency and other figures are calculated on the device from the data each provider returns and from a price table that can go out of date. Some values are estimated. Providers change prices, rounding, caching and billing rules without notice.

Always cross-check

Before you make a business, billing or budgeting decision, compare the numbers with your provider's own usage dashboard and invoices. The provider's figures are the ones you are charged for.

No warranty

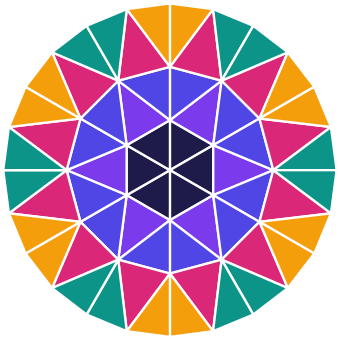
The software, this guide and the website are provided "as is", without warranty of any kind, as stated in the BSD 3-Clause licence. You are responsible for your API keys, your users' data, your privacy notices and your

compliance with each provider's terms.

Full terms

The complete Terms and Conditions are in `TERMS.md` in the repository and on the website under Legal. If this summary and the full Terms differ, the full Terms apply.

kaleidoscope
DEVELOPED BY SANKET SANTOSH VIDYA MANOHAR



kaleidoscope

AI usage analytics for Flutter

DEVELOPED BY

Sanket Santosh Vidya Manohar

Package	pub.dev/packages/kaleidoscope
Source	github.com/sanket-manohar/kaleidoscope
Docs	sanket-manohar.github.io/kaleidoscope
Website	sanketmanohar.com
pub.dev	pub.dev/publishers/sanketmanohar.com
VS Code	marketplace.visualstudio.com/publishers/sanket-manohar
YouTube	youtube.com/@sanketmanoharstudio
Instagram	instagram.com/sanketmanoharstudio

kaleidoscope 1.0.0 · Released under the BSD 3-Clause licence
© 2026 Sanket Santosh Vidya Manohar
Not affiliated with or endorsed by any AI provider. All product names and brands belong to their owners.